Architectural Guide to JSON Web Tokens (JWT)

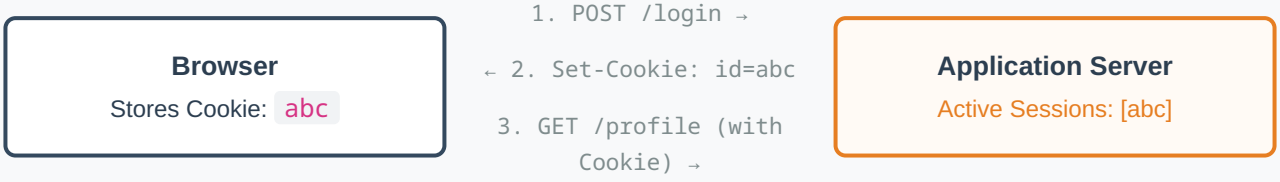
STATEFUL VS. STATELESS AUTHENTICATION & TOKEN LIFECYCLE MANAGEMENT

DOCUMENT TYPE	TARGET AUDIENCE	TOPICS COVERED
Technical Reference Manual	Backend & Systems Engineers	Session Auth, JWT Anatomy, Security

1. Stateful Authentication (Session-Based)

Historically, web applications utilized **Stateful Authentication** to manage user state. In this paradigm, the identity and authentication status of a user are explicitly tracked and maintained on the server side.

FIGURE 1.1: STATEFUL SESSION ARCHITECTURE



The Technical Workflow:

1. **Credentials Verification:** The client sends a username and password to the server via a secure request.
2. **Session Creation:** Upon validation, the server generates a unique identifier (e.g., `sessionId = "abc"`) and reserves a slot in its memory/database to store session details.
3. **Cookie Transmission:** The identifier is passed back to the browser via the `Set-Cookie` header. The client automatically stores this cookie and appends it to all subsequent requests.

Core Architectural Bottlenecks of Stateful Systems

- **Server Volatility & Restarts:** Because sessions reside in server memory, restarting an application instance instantly invalidates all active sessions, causing an immediate mass logout of all active users.
- **Horizontal Scaling Challenges:** When multiple server instances run behind a standard Load Balancer, a client request might hit Server A (where the session exists) on request 1, but hit Server B (where the session does not exist) on request 2. This requires complex mitigations like *Sticky Sessions* or externalized caching layers (e.g., Shared Redis clusters).

2. Stateless Authentication (Token-Based)

To mitigate horizontal scaling complexities, modern distributed architectures heavily leverage **Stateless Authentication**. Under this mechanism, the server maintains *zero state* regarding logged-in users. Instead, all identity details are contained securely inside an encrypted or cryptographically signed token held exclusively by the client.

FIGURE 2.1: STATELESS TOKEN VERIFICATION ARCHITECTURE

When a request arrives at any server instance behind a load balancer, the instance applies a mathematical cryptographic verification process using a shared **Secret Key**. If the signature matches, the server trusts the contents explicitly without querying an active session database.

3. Anatomy of a JSON Web Token (JWT)

A JSON Web Token is represented structurally as a single string split into three distinct segments delimited by dots (`.`). Each segment represents a base64url-encoded JSON object.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI5IiwiZW1haWwiOiJyYWtlc2hAY29kZXJzZ3lhbi5jb20iLCJpYXQiOjE3NjAwMDk2MDcsImV4cCI6MTc2MDAxMDIwN30.XsQVvWT5Ekfag01RqEvg2d9sPQN8iEB0FnRZO3qltfw
```

1. HEADER

Defines the metadata for the token, explicitly specifying the signing algorithm and token format type.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. PAYLOAD

Contains the application assertions ("claims"), such as unique user identifiers, metadata, and token lifecycle timestamps.

```
{
  "sub": "9",
  "email": "rakesh@...",
  "iat": 1760009607,
  "exp": 1760010207
}
```

3. SIGNATURE

Calculated by taking the encoded header, the encoded payload, combining them with a server-side secret key, and passing them to the specified algorithm.

```
HMACSHA256(
  base64Url(header) + "." +
  base64Url(payload),
  secret_key
)
```

Critical Cryptographic Fact: Visibility vs. Tampering

Base64Url encoding is **NOT encryption**. The payload data is visible to anyone who intercepts the token string. The security guarantees of JWT rely entirely on the **Signature** segment: if a malicious actor alters the payload content to elevate privileges, the recalculated signature will mismatch the token's signature segment, forcing the verification step to fail immediately.

4. Advanced Token Lifecycle: Access vs. Refresh

Deploying stateless tokens introduces an inherent security tradeoff: once a token is dispatched, the server cannot naturally revoke it or pull it back. If an access token with a 1-year lifespan is compromised, an attacker can exploit it unchecked for that full year.

To solve this dilemma, production systems isolate functionality into a dual-token paradigm:

- **Access Token:** Extremely short lifespan (e.g., 15 minutes or 60 seconds). It is optimized for speed and verified entirely in-memory by resource servers.
- **Refresh Token:** Long lifespan (e.g., 30 days to 1 year). It is securely stored in a safe client context and used exclusively to request fresh Access Tokens from the auth server when the previous one expires.

FIGURE 4.1: ACCESS EXPIRATION & REFRESH FLOW

Client Browser	Network Interface / Actions	Application Server
Holds expired token	1. Requests API with Expired Access Token →	**401 Unauthorized**
Initiates Refresh Flow	2. Sends Valid Refresh Token to /refresh endpoint →	Validates & issues fresh pairs
Saves new access token	← 3. Returns New Short-Lived Access Token	Session Extended Seamlessly

Handling the Stateless Logout Challenge

Because verification does not read database records, simply deleting an access token from the browser client cache during logout does not stop an attacker who already possesses a copied token from calling the system APIs.

To handle explicit **Logouts** safely:

1. The client hits a `/logout` endpoint, which explicitly wipes out or blacklists the **Refresh Token** from the server database, ensuring no additional Access Tokens can ever be generated.
2. For ultimate stringency, active access tokens can be flagged in an ultra-fast temporary in-memory data warehouse (like Redis) as blacklisted items until their brief natural expiration timer runs out.